Tutorial 3

Comparing objects · Spaceship operator · auto · Default <=>

Review: Operator Overloading

Last week we saw that operators can be given meaning for custom types:

```
struct Vec { int x, y; };

Vec operator+(const Vec &v1, const Vec &v2) {
    return { v1.x + v2.x, v1.y + v2.y };
}

ostream &operator<<(ostream &out, const Grade &g) {
    out << g.theGrade << '%';
    return out;   // enables chaining
}
```

- Must return the stream by **reference** to support chaining
- Today: overloading the **comparison** operators

This week

1. Comparing objects in C vs C++
2. The three-way comparison (spaceship) operator <=>
3. `auto` type deduction
4. Defaulted <=> and getting 6 operators for free
5. When the default is wrong

In C, string comparison requires a function call:

```
strcmp(s1, s2)   // < 0 if s1 < s2,  0 if equal,  > 0 if s1 > s2
```

- Linear scan, char-by-char, lexicographic order

In C++, you can write it naturally:

```
if (s1 < s2) { ... }
else if (s1 == s2) { ... }
else { ... }
```

C++ version is more readable — but notice the **hidden cost**: `s1` and `s2` are compared **twice**.

With `strcmp`, one call tells you everything:

```
int n = strcmp(s1, s2);   // ONE comparison
if (n < 0) { ... }
else if (n == 0) { ... }
else { ... }
```

With C++ < and ==, you pay for **two full scans** of the strings:

```
if (s1 < s2) { ... }         // scan #1
else if (s1 == s2) { ... }  // scan #2 — redundant!
```

Can we get the best of both worlds — readable syntax **and** a single comparison?

Yes! C++20 introduces the **three-way comparison operator** <=>:

```
import <compare>;

string s1 = ..., s2 = ...;
std::strong_ordering result = s1 <=> s2;    // ONE comparison

if (result < 0)  cout << "less";
else if (result == 0) cout << "equal";
else             cout << "greater";
```

Result	Meaning
`result < 0`	`s1` is less than `s2`
`result == 0`	`s1` equals `s2`
`result > 0`	`s1` is greater than `s2`

`std::strong_ordering` is a lot to type. Use `auto`:

```
auto result = s1 <=> s2;   // compiler deduces the type
```

`auto x = expr;` declares `x` with the same type as the value of `expr`.

You've already seen this with range-for loops:

```
for (auto x : vec) { ... }
```

Using `auto` here is idiomatic — the exact ordering type (`strong_ordering`, `weak_ordering`, `partial_ordering`) depends on the types being compared.

You can overload <=> as a member function:

```
struct Vec {
    int x, y;

    auto operator<=>(const Vec &other) const {
        auto n = x <=> other.x;
        return (n == 0) ? (y <=> other.y) : n;
    }
};
```

Comparison is lexicographic on `(x, y)`:

- Compare `x` first; if equal, compare `y`

```
Vec v1{1, 2}, v2{1, 3};
auto r = v1 <=> v2;   // r < 0, because 2 < 3
```

Once <=> is defined, C++ **automatically rewrites** the six relational operators in terms of it:

You write	Compiler rewrites as
`v1 <= v2`	`(v1 <=> v2) <= 0`
`v1 < v2`	`(v1 <=> v2) < 0`
`v1 == v2`	`(v1 <=> v2) == 0`
`v1 != v2`	`(v1 <=> v2) != 0`
`v1 > v2`	`(v1 <=> v2) > 0`
`v1 >= v2`	`(v1 <=> v2) >= 0`

Define <=> once → get <, <=, ==, !=, >, >= for free!

For simple structs, you can ask the compiler to write <=> for you:

```
struct Vec {
    int x, y;
    auto operator<=>(const Vec &other) const = default;
    // lexicographic order on fields, left to right:
    // compare x first, then y — same as what we wrote manually
};
```

`= default` generates member-wise lexicographic comparison automatically.

This is equivalent to the hand-written version on the previous slide.

The default compares **field values directly** — which breaks for pointer fields:

```
struct Node {
    int data;
    Node *next;   // default <=> would compare pointer ADDRESSES — useless!
};
```

We want to compare the **contents** of the linked list, not raw memory addresses.

Rule of thumb: `= default` is safe when all fields are value types (`int`, `string`, …). When the class **manages memory** (has pointer fields), write `<=>` by hand.

4 · Hand-Written <=> for Linked Lists


```
struct Node {
    int data;
    Node *next;

    auto operator<=>(const Node &other) const {
        auto n = data <=> other.data;
        if (n != 0) return n;
        // both null → equal
        if (!next && !other.next) return n;
        // one null → shorter list is "less"
        if (!next)       return std::strong_ordering::less;
        if (!other.next) return std::strong_ordering::greater;
        // recurse into the rest of the list
        return *next <=> *other.next;
    }
};
```

Common Pitfalls — Quick Reference

Note for previous slide: assumes non-empty lists; handle the empty-list case separately in practice.

Mistake	**Fix**
Using < and == in the same if-else chain	Use <=> to do a single comparison
Forgetting `import <compare>`	Required for `std::strong_ordering`
Using `= default` when class has pointer fields	Write <=> manually; default compares addresses
<=> returns the wrong ordering type	Use `auto` as the return type
Comparing a null `next` pointer via recursion	Check for `nullptr` before dereferencing

This week's problem set

```
const string RANKS = "3456789TJQKA2";
const string SUITS = "DCHS";

struct Card {
    char rank;  // one of the characters in RANKS
    char suit;  // one of the characters in SUITS

    // Fill in the return type
    ___ operator<=>(const Card& other) const {
      // Fill in this operator overload
    }
};
```

Use `RANKS.find('4') = 1` to find the index of a character in a string. Comparing the characters directly will not work: for example, 'K' < 'T' in ASCII, but a king is higher than a ten.

Important Reminders:

- Due date for part 2: **Wednesday, Oct 14, 5 pm**

Refer to Piazza first for questions and pull the most recent files.